Cuaderno de Ataques y Pruebas de Seguridad

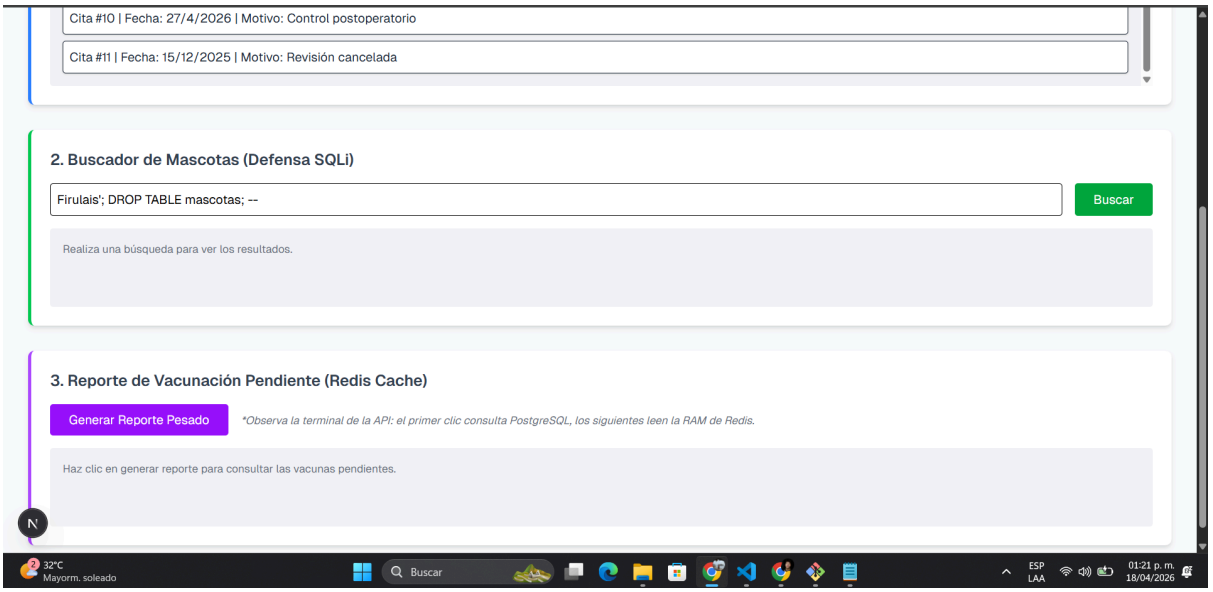
Materia: Base de Datos Avanzadas **Alumno:** Edwar Daniel González Hernández

---

Sección 1: Tres ataques de SQL Injection que fallan

Ataque 1: Quote-escape clássico

- **Input exacto probado:** `OR '1'='1`
- **Superficie atacada:** Campo de texto en la sección "Buscador de Mascotas" del Frontend.
- **Evidencia del Frontend:**



- **Evidencia de Falla (Log):**

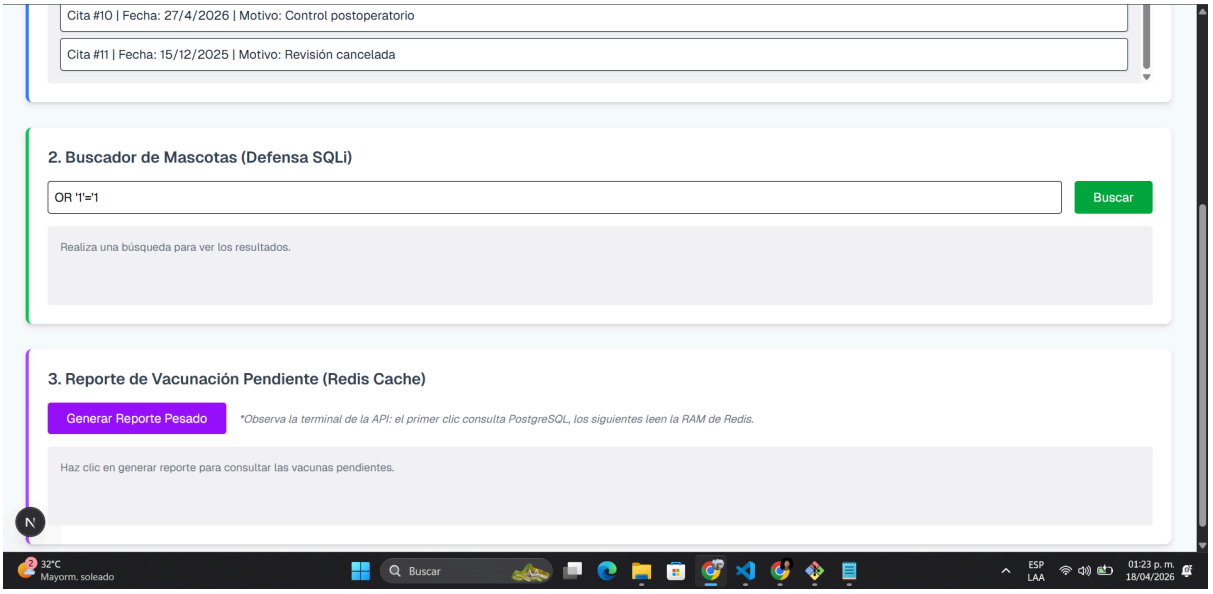
```
MINGW64:/c/Users/USER/OneDrive/Documentos/db_avanzada/corte3-bda-{2...
uthentication and is not protected by network restrictions. Redis will accept co
nnections from any IP address on any network interface.
vet-api        | ◇ injected env (5) from .env // tip: ⌘ enable debugging { debug:
 true }
vet-api        | API corriendo en http://localhost:3000
vet-api        | Conectado exitosamente a Redis
vet-frontend   |
vet-frontend   | > clinica-veterinaria@0.1.0 dev
vet-frontend   | > next dev
vet-frontend   |
vet-frontend   | ▲ Next.js 16.2.4 (Turbopack)
vet-frontend   | - Local:         http://localhost:3000
vet-frontend   | - Network:       http://172.28.0.5:3000
vet-frontend   | ✓ Ready in 572ms
vet-frontend   |
vet-frontend   |  GET / 200 in 4.8s (next.js: 4.3s, application-code: 520ms)
vet-api        | [API] GET /api/citas -> Estado: 304
vet-api        | Petición de búsqueda recibida: "Firulais'; DROP TABLE mascotas;
--"
vet-api        | [API] GET /api/mascotas/buscar?nombre=Firulais%27;%20DROP%20TABL
E%20mascotas;%20-- -> Estado: 404

v View in Docker Desktop   o View Config   w Enable Watch
```

- **Línea exacta de defensa:** Archivo `api/index.js` (línea 118). La línea es: `const values = ["%" + nombre + "%"]; const result = await pool.query(query, values);`

Ataque 2: Stacked query (Intento de destrucción)

- **Input exacto probado:** `Firulais'; DROP TABLE mascotas; --`
- **Superficie atacada:** Campo de texto en la sección "Buscador de Mascotas" del Frontend.
- **Evidencia del Frontend:**



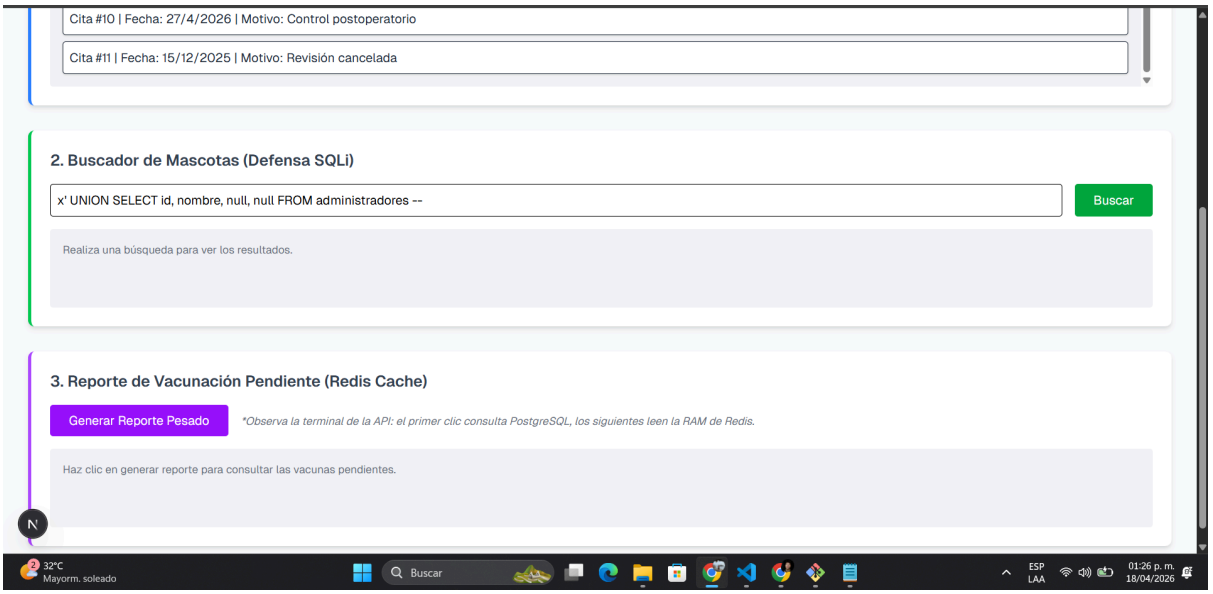
- **Evidencia de Falla (Log):**

- **Línea exacta de defensa:** Archivo `api/index.js` (Aprox. línea 106). Se utiliza el driver `pg` con consultas parametrizadas (`$1`), lo cual impide apilar comandos ejecutables.

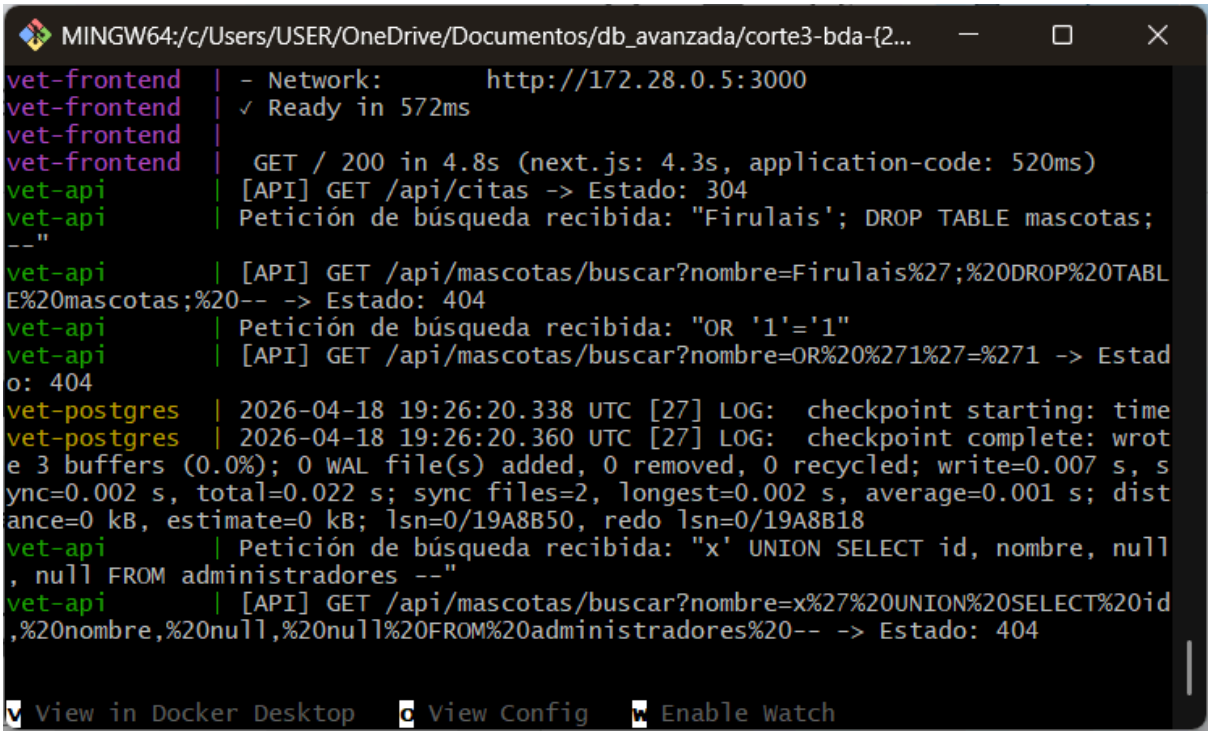
Ataque 3: Inyección basada en UNION

- **Input exacto probado:** `x' UNION SELECT id, nombre, null, null FROM administradores --`

- **Superficie atacada:** Campo de texto en la sección "Buscador de Mascotas" del Frontend.
- **Evidencia del Frontend:**



- **Evidencia de Falla (Log):**



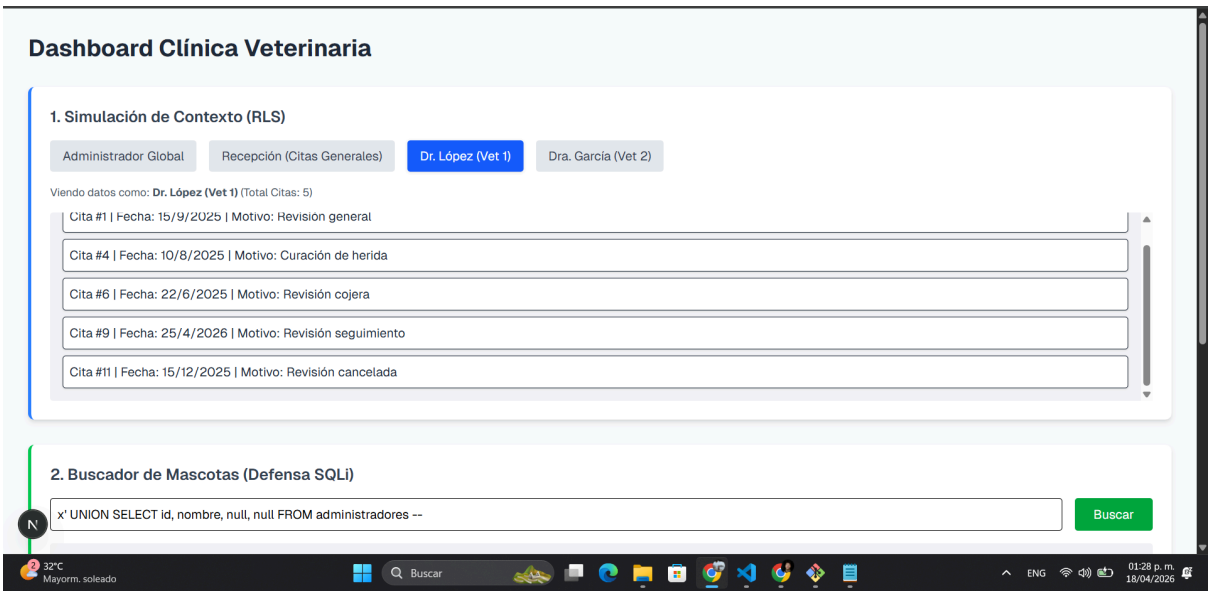
- **Línea exacta de defensa:** Archivo `api/index.js` (Aprox. línea 106). La parametrización estricta trata todo el input como un string literal.

Sección 2: Demostración de Row-Level Security (RLS) en acción

Contexto: Se prueba el sistema con dos veterinarios distintos, cada uno con mascotas diferentes asignadas.

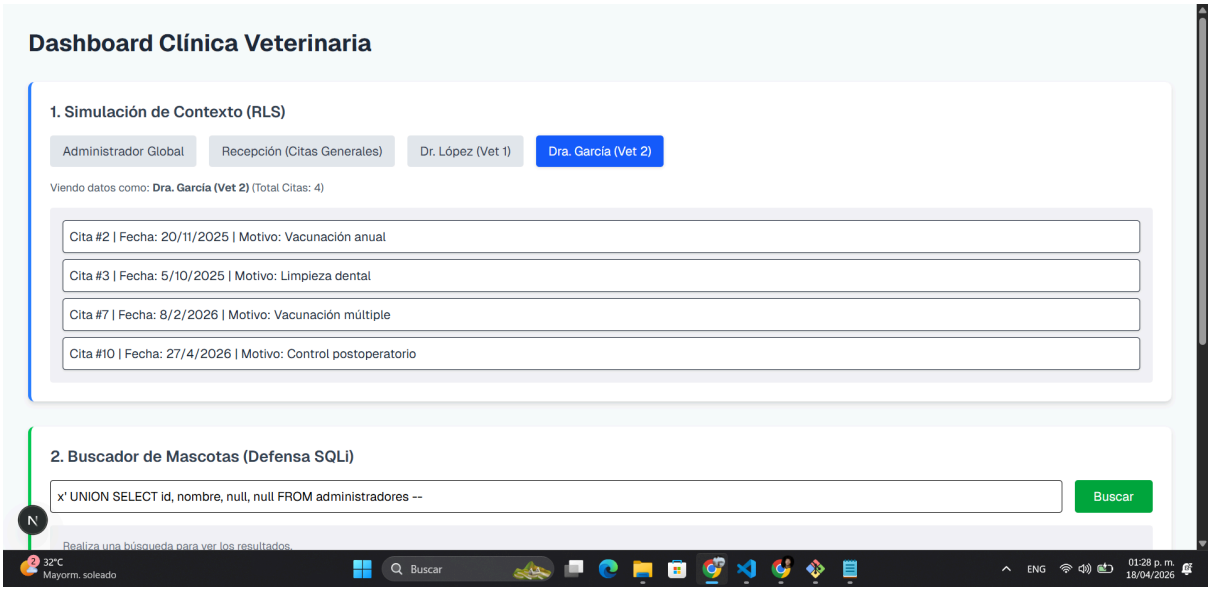
Vista del Veterinario 1 (Dr. López)

- Al ingresar con el rol del Dr. López, el sistema devuelve **solo sus 5 citas** asignadas.
- **Evidencia:**



Vista del Veterinario 2 (Dra. García)

- Al cambiar al rol de la Dra. García, la lista cambia dinámicamente y ella visualiza **solo sus 4 citas** asignadas.
- **Evidencia:**



Explicación de la Política: Este aislamiento de datos es producto de la política RLS: `CREATE POLICY rls_vet_citas ON citas FOR SELECT TO rol_veterinario USING (veterinario_id = current_setting('app.current_vet_id')::INT);`. Esta política filtra las filas desde el motor de la base de datos comparando el dueño de la cita con el ID inyectado por el backend en la sesión actual.

Sección 3: Demostración de Caché Redis funcionando

Evidencia de Logs de Terminal:

```
MINGW64:/c/Users/USER/OneDrive/Documentos/db_avanzada/corte3-bda-{2...
vet-redis      | 1:M 18 Apr 2026 19:46:29.696 * DB loaded from disk: 0.000 second
s
vet-redis      | 1:M 18 Apr 2026 19:46:29.696 * Ready to accept connections tcp
vet-redis      | 1:M 18 Apr 2026 19:46:29.696 # WARNING: Redis does not require a
uthentication and is not protected by network restrictions. Redis will accept co
nnections from any IP address on any network interface.
vet-frontend   | ▲ Next.js 16.2.4 (Turbopack)
vet-frontend   | - Local:         http://localhost:3000
vet-frontend   | - Network:       http://172.28.0.5:3000
vet-frontend   | ✓ Ready in 599ms
vet-frontend   |
vet-frontend   |  GET / 200 in 2.4s (next.js: 1696ms, application-code: 723ms)
vet-api        | [API] GET /api/citas -> Estado: 304
vet-api        | Sirviendo desde PostgreSQL
vet-api        | [API] GET /api/reportes/vacunacion-pendiente -> Estado: 304
vet-api        | Sirviendo desde Redis Cache
vet-api        | [API] GET /api/reportes/vacunacion-pendiente -> Estado: 304
vet-api        | 🗑 Invalidando caché de vacunaciones pendientes...
vet-api        | [API] POST /api/vacunas -> Estado: 201
vet-api        | Sirviendo desde PostgreSQL
vet-api        | [API] GET /api/reportes/vacunacion-pendiente -> Estado: 304

View in Docker Desktop   View Config   Enable Watch
```

Explicación de la Estrategia:

- **Llave (Key) utilizada:** `reporte:vacunaciones_pendientes`. Esta llave identifica unívocamente al reporte general.
- **TTL elegido:** 60 segundos.
- **Justificación:** Al ser un reporte pesado, 60 segundos es el balance ideal. Protege a PostgreSQL de picos de lectura simultáneos de múltiples recepcionistas, pero garantiza que la información se refresque razonablemente rápido. Además, el caché se **invalida automáticamente** al ejecutarse un `POST` a `/api/vacunas`, asegurando la consistencia inmediata si ocurre un cambio crítico antes de que expire el TTL.